KATEDRA
INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI

Paradigmata programování 2

# 4 Mutace

Michal Krupka

# Obsah


1 Změna hodnoty vazby symbolu

2 Mutace v datových strukturách

3 Místa

4 Implementace míst

5 Generátory

6 Kruhové datové struktury

# Změna hodnoty vazby symbolu

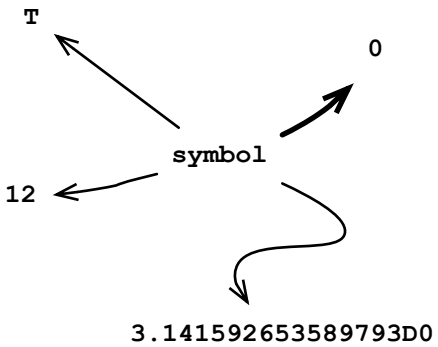
Nastavení hodnoty vazby symbolu výrazem

```
(setq symbol (+ symbol 5))
```

# Změna hodnoty vazby symbolu

Nastavení hodnoty vazby symbolu výrazem

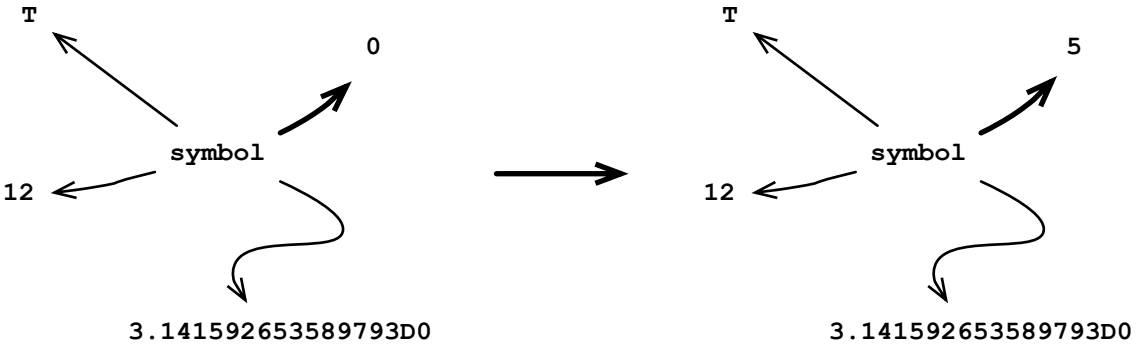
```
(setq symbol (+ symbol 5))
```



# Změna hodnoty vazby symbolu

Nastavení hodnoty vazby symbolu výrazem

```
(setq symbol (+ symbol 5))
```



# Mutace v lexikálních uzávěrech

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (bind list1 (two-functions 0))
```

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (bind list1 (two-functions 0))
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 0 |

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (bind list1 (two-functions 0))
(#<ANONYMOUS-FUNCTION ...> #<ANONYMOUS-FUNCTION ...>)
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 0 |

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (first list1))
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 0 |

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (first list1))
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 0 |

Prostředí první funkce

| symbol | hodnota |
|---|---|

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (first list1))
0
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 0 |

Prostředí první funkce

| symbol | hodnota |
|---|---|

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (second list1) 1)
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 0 |

Prostředí první funkce

| symbol | hodnota |
|---|---|

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (second list1) 1)
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 0 |

Prostředí první funkce

| symbol | hodnota |
|---|---|

Prostředí druhé funkce

| symbol | hodnota |
|---|---|
| y | 1 |

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (second list1) 1)
1
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 1 |

Prostředí první funkce

| symbol | hodnota |
|---|---|

Prostředí druhé funkce

| symbol | hodnota |
|---|---|
| y | 1 |

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 1 |

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (first list1))
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 1 |

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (first list1))
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 1 |

Prostředí první funkce

| symbol | hodnota |
|---|---|

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

```
> (funcall (first list1))
1
```

Prostředí funkce `two-functions`

| symbol | hodnota |
|---|---|
| x | 1 |

Prostředí první funkce

| symbol | hodnota |
|---|---|

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

# Příklad

Dvě důležité skutečnosti:

# Příklad

Dvě důležité skutečnosti:

- prostředí funkce `two-functions` nikdy nezaniká

# Příklad

Dvě důležité skutečnosti:

- prostředí funkce `two-functions` nikdy nezaniká
- při aplikaci funkce vzniká pokaždé nové prostředí

# Příklad

Dvě důležité skutečnosti:

- prostředí funkce `two-functions` nikdy nezaniká
- při aplikaci funkce vzniká pokaždé nové prostředí

Prostředí funkce `two-functions` (první aplikace)

| symbol | hodnota |
|---|---|
| x | 1 |

Prostředí funkce `two-functions` (druhá aplikace)

| symbol | hodnota |
|---|---|
| x | a |

Obě prostředí můžeme používat současně (přes proměnné `list1` a `list2`).

# Příklad: tečkové páry s mutátory

# Příklad: tečkové páry s mutátory

```
(defun my-cons (x y)
  (lambda (what &optional val)
    (cond ((eql what 'car) x)
          ((eql what 'cdr) y)
          ((eql what 'set-car) (setf x val))
          ((eql what 'set-cdr) (setf y val)))))
```

```
(defun my-car (c)
  (funcall c 'car))

(defun my-cdr (c)
  (funcall c 'cdr))
```

```
(defun my-set-car (c val)
  (funcall c 'set-car val))

(defun my-set-cdr (c val)
  (funcall c 'set-cdr val))
```

# Obsah


1 Změna hodnoty vazby symbolu

2 Mutace v datových strukturách

3 Místa

4 Implementace míst

5 Generátory

6 Kruhové datové struktury

# Mutace v párech a seznamech

# Mutace v párech a seznamech

- mutace v párech: funkce `set-car` a `set-cdr`
- pozor na totožnost (identitu) párů

# Pozor

# Pozor

Páry lze modifikovat, pouze když vznikly za běhu programu.
Toto je zakázáno:

```
> (bind p '(1 . 2))
(1 . 2)

> (set-car p 2)                                    nesprávně
```

Dále není povoleno modifikovat seznamy v `&rest`-parametrech.

# Obsah


1 Změna hodnoty vazby symbolu

2 Mutace v datových strukturách

3 Místa

4 Implementace míst

5 Generátory

6 Kruhové datové struktury

# Místa

V jiných jazycích: příkaz přiřazení

# Makro `setf`

# Makro `setf`

```
> (bind a 0)
0

> (setf a 1)
1

> a
1

> (bind pair (cons 1 2))
(1 . 2)

> (setf (car pair) 2)
2

> pair
(2 . 2)
```

# Makro `incf`

# Makro incf

```
> (incf a)
2

> a
2

> (incf a 8)
10

> a
10

> (incf (car pair) -2)
0

> pair
(0 . 2)
```

# Místa

# Místa

```
(setf place expression) => new-value
```

obecně:

```
(setf place1 expression1 ... placen expressionn) => new-value-n
```

nebo:

```
(incf place) => new-value
```

*place*: místo

# Implementace míst: přístupové funkce

Každé místo má dvě přístupové funkce: selektor a mutátor

získávají se makrem `place-accessors`

# Makro `place-accessors`: použití

```
(place-accessors a)        ⟶  (list (lambda () a)
                                    (lambda (val)
                                      (setq a val)))

(place-accessors           ⟶  (let ((p pair))
  (car pair))                  (list (lambda () (car p))
                                     (lambda (val)
                                       (set-car p val)
                                       val)))

(place-accessors           ⟶  (let ((p (nth n list)))
  (cdr (nth n list)))          (list (lambda () (cdr p))
                                     (lambda (val)
                                       (set-cdr p val)
                                       val)))
```

# Makra pracující s místy

- `setf`
- `incf`
- `push`, `pop`
- ...

# Obsah


1 Změna hodnoty vazby symbolu

2 Mutace v datových strukturách

3 Místa

4 Implementace míst

5 Generátory

6 Kruhové datové struktury

# symbol-accessors

# symbol-accessors

```
(defmacro symbol-accessors (symbol)
  `(list (lambda () ,symbol)
         (lambda (val) (setq ,symbol val))))
```

# car-accessors

# car-accessors

```
(defmacro car-accessors (pair-expr)
  `(let ((p ,pair-expr))
     (list (lambda () (car p))
           (lambda (val)
             (set-car p val)
             val))))
```

# place-accessors

# place-accessors

```
(defmacro place-accessors (place)
  (if (symbolp place)
      `(symbol-accessors ,place)
    `(,(symbol (stra (car place) "-ACCESSORS")) ,@(cdr place))))
```

# Obsah


1 Změna hodnoty vazby symbolu

2 Mutace v datových strukturách

3 Místa

4 Implementace míst

5 Generátory

6 Kruhové datové struktury

# Obsah


1 Změna hodnoty vazby symbolu

2 Mutace v datových strukturách

3 Místa

4 Implementace míst

5 Generátory

6 Kruhové datové struktury

# Místa v seznamech

. . . zdrojový kód

# Kruhové seznamy

# Kruhové seznamy

```
> (bind result (list 1))
> (setf (cdr result) result)
```

# Kruhové seznamy

```
> (bind result (list 1))
> (setf (cdr result) result)
```

Můžeme napsat i funkci:

# Kruhové seznamy

```
> (bind result (list 1))
> (setf (cdr result) result)
```

Můžeme napsat i funkci:

```
(defun cl (elem)
  (let ((result (list elem)))
    (setf (cdr result) result)
    result))
```

# Kruhové seznamy

```
> (bind result (list 1))
> (setf (cdr result) result)
```

Můžeme napsat i funkci:

```
(defun cl (elem)
  (let ((result (list elem)))
    (setf (cdr result) result)
    result))
```

Hodnota `(cl 1)`:

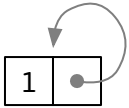
# Kruhové seznamy

```
> (bind result (list 1))
> (setf (cdr result) result)
```

Můžeme napsat i funkci:

```
(defun cl (elem)
  (let ((result (list elem)))
    (setf (cdr result) result)
    result))
```

Hodnota `(cl 1)`:



# Kruhové seznamy: nové definice

# Kruhové seznamy: nové definice

Nová definice seznamu

Seznam je buď symbol `nil`, nebo pár.

Tečkový seznam je seznam *list* takový, že existuje číslo *n*, pro které (`nthcdr` *n list*) není seznam.

Kruhový seznam je seznam *list* takový, že existují dvě různá čísla *m* a *n*, pro která (`nthcdr` *m list*) ani (`nthcdr` *n list*) není `nil` a současně (`eql` (`nthcdr` *m list*) (`nthcdr` *n list*)) je *Pravda*, tj. není `nil`.

Čistý (spojový) seznam je seznam, který není ani tečkový, ani kruhový.

Pokud řekneme *seznam*, myslíme tím pouze čistý seznam.

# Test kruhovosti (netečkového) seznamu

# Test kruhovosti (netečkového) seznamu

Rychlý a pomalý běžec

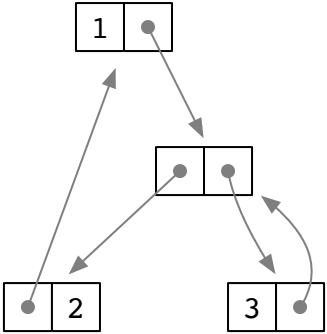
# Test kruhovosti (netečkového) seznamu

Rychlý a pomalý běžec

```
(defun circlep (list)
  (labels ((cp (slow fast)
             (cond ((null fast) nil)
                   ((eql slow fast) t)
                   (t (cp (cdr slow) (cddr fast))))))
    (cp (cdr list) (cddr list))))
```

# Kruhová struktura: příklad

# Kruhová struktura: příklad



# Obecná datová struktura

- kruhová (cyklická)
- nekruhová (acyklická)
- stromová

# Test stromovosti

# Test stromovosti

```
(defun treep (s)
  (let ((visited '()))
    (labels ((tp (node)
               (cond ((find node visited) nil)
                     ((consp node)
                      (push node visited)
                      (and (tp (car node))
                           (tp (cdr node))))
                     (t t))))
      (cs s))))
```